

# ESTUDIO — Carpeta exceptions/

Material de estudio para el final del TFC Bajoneá. Basado en el código real de `backend/src/main/java/com/bajonea/backend/exceptions/` — **4 excepciones custom + 1 handler global**.

## La idea general, primero

Cuando algo sale mal en el backend, hay dos formas de manejarlo:

**La mala:** llenar cada método de `if` y `try/catch`, y que cada uno arme su respuesta de error a mano. Resultado: código repetido en todos lados y mensajes de error distintos según quién los escribió.

**La que usa Bajoneá:** el service **lanza una excepción** y se olvida. Una sola clase central —el `GlobalExceptionHandler` — la atrapa y la traduce al código HTTP y al mensaje que corresponde.

```
// En el service, así de simple:
throw new ConflictoDeNegocioException("Ya existe una cuenta con ese email");

// El handler lo convierte solo en:
// HTTP 409 → {"mensaje": "Ya existe una cuenta con ese email", "data": null}
```

**La ventaja para contar en la mesa:** la lógica de negocio queda limpia (no sabe nada de HTTP), y el formato de error es idéntico en toda la API. Si mañana querés cambiar cómo se ven los errores, tocás un solo archivo.

## Un concepto previo: por qué todas extienden RuntimeException

En Java hay dos familias de excepciones:

|                      | <i>Checked</i> ( <code>extends Exception</code> ) | <i>Unchecked</i> ( <code>extends RuntimeException</code> ) |
|----------------------|---------------------------------------------------|------------------------------------------------------------|
| ¿Hay que declararla? | Sí, con <code>throws</code> en la firma           | No                                                         |
| ¿Hay que atraparla?  | Sí, obligatorio                                   | No                                                         |
| Ejemplo              | <code>IOException</code>                          | <code>NullPointerException</code>                          |

Las 4 excepciones del proyecto son **unchecked**. Si fueran checked, cada método de service tendría que declarar `throws ConflictoDeNegocioException`, y cada controller tendría que envolverlas en `try/catch`. Ensuciaría todo el código para nada, porque igual las maneja el handler.

**Y hay un motivo técnico más:** Spring hace **rollback automático de la transacción** solo con excepciones *unchecked*. Si fueran checked, un error a mitad de un pedido dejaría datos guardados a medias.

# LAS 4 EXCEPCIONES CUSTOM

## 1. RecursoNoEncontradoException → 404 Not Found

### Cuándo se dispara

Cuando pedís algo **que no existe**.

```
Producto producto = productoRepository.findById(id)
    .orElseThrow(() -> new RecursoNoEncontradoException("Producto no encontrado"));
```

### Qué información lleva

Solo un `String` con el mensaje. Es la más simple de las cuatro.

### Cómo la maneja el handler

```
@ExceptionHandler(RecursoNoEncontradoException.class)
→ HTTP 404 + ApiResponse(mensaje de la excepción, null)
```

### Ejemplos reales

- `GET /productos/9999` cuando ese producto no existe.
- Buscar un comercio por un id inválido.
- Marcar como leída una notificación inexistente.

## 2. ConflictDeNegocioException → 409 Conflict

### Cuándo se dispara

Cuando el pedido está bien formado, pero choca contra una regla de negocio o contra el estado actual del sistema.

### Qué información lleva

Un `String` con el mensaje.

### Cómo la maneja el handler

```
@ExceptionHandler(ConflictDeNegocioException.class)
→ HTTP 409 + ApiResponse(mensaje, null)
```

### Los casos reales del proyecto

| Situación                                                    | Mensaje típico                              |
|--------------------------------------------------------------|---------------------------------------------|
| Email ya registrado                                          | "Ya existe una cuenta con ese email"        |
| Producto de otro comercio en el carrito                      | El carrito solo admite un comercio a la vez |
| Sexta imagen de un producto                                  | El límite es 5                              |
| Login con cuenta bloqueada o inactiva                        | La cuenta no está en condiciones de operar  |
| Actuar sobre un pedido que ya no está <code>PENDIENTE</code> | Ya fue aceptado o rechazado                 |

### La diferencia 400 vs 409 — pregunta muy probable

|                        | 400 Bad Request                       | 409 Conflict                                           |
|------------------------|---------------------------------------|--------------------------------------------------------|
| Qué significa          | "Lo que me mandaste está mal formado" | "Lo que me mandaste está bien, pero no se puede hacer" |
| Ejemplo                | Un email sin <code>@</code>           | Un email perfecto, pero ya registrado                  |
| Depende de             | El dato en sí                         | El <b>estado</b> del sistema                           |
| ¿Cambia con el tiempo? | No: siempre va a estar mal            | Sí: mañana ese email podría estar libre                |

Esa última fila es la mejor forma de explicarlo: un 400 nunca va a funcionar por más que reintentés; un 409 podría funcionar en otro momento.

## 3. ValidacionException → 400 Bad Request

### Cuándo se dispara

Para validaciones de negocio que **no se pueden expresar como una anotación en el DTO**.

### Por qué existe si ya está Bean Validation

Bean Validation ( `@NotBlank` , `@Email` , etc.) tiene dos límites concretos:

1. **Valida un campo a la vez**. No puede expresar "este campo es obligatorio solo si aquel otro vale X".
2. **No puede consultar la base**. No sabe si un id existe.

Los casos reales del proyecto que caen en `ValidacionException` :

| Caso                                                                                | Por qué no puede ir en el DTO                                   |
|-------------------------------------------------------------------------------------|-----------------------------------------------------------------|
| <code>direccionId</code> obligatorio solo si <code>tipoEntrega = DOMICILIO</code>   | Depende de otro campo.                                          |
| <code>motivo</code> obligatorio solo si <code>aprobar = false</code>                | Depende de otro campo.                                          |
| <code>horaCierre</code> posterior a <code>horaApertura</code>                       | Compara dos campos entre sí.                                    |
| El comercio debe ofrecer <code>delivery</code> o <code>retiro</code> (al menos uno) | Compara dos campos. Está en <code>ComercioValidaciones</code> . |
| Una FK que apunta a un id que no existe                                             | Requiere consultar la base.                                     |

### Cómo la maneja el handler

```
@ExceptionHandler(ValidacionException.class)
→ HTTP 400 + ApiResponse(mensaje, null)
```

**Fijate que devuelve el mismo 400 que Bean Validation**. Es a propósito: para el frontend son ambos "el usuario mandó algo inválido", sin importar dónde se detectó.

## 4. CredencialesInvalidasException → 401 Unauthorized

### Cuándo se dispara

Cuando fallan las credenciales o un código de un solo uso:

- Email o contraseña incorrectos en el login.
- La `passwordActual` no coincide al cambiar la contraseña desde el perfil.
- Un token de verificación / recuperación / reactivación inválido, expirado o ya usado.

### Qué información lleva — es la única distinta

```
public class CredencialesInvalidasException extends RuntimeException {
    private final Object data;

    public CredencialesInvalidasException(String mensaje) { this(mensaje, null); }
    public CredencialesInvalidasException(String mensaje, Object data) {
        super(mensaje);
        this.data = data;
    }
    public Object getData() { return data; }
}
```

Es la única que puede llevar un `data` extra, además del mensaje. Tiene dos constructores: uno solo con mensaje, otro con mensaje + datos.

Para qué sirve ese `data` : para mandarle contexto útil al frontend. Por ejemplo, cuántos intentos le quedan al usuario antes de que se le bloquee la cuenta. El handler lo pasa tal cual:

```
→ HTTP 401 + ApiResponse(mensaje, ex.getData())
```

### El bug real del @Transactional — muy buena anécdota para la mesa

Al implementar el bloqueo por 3 intentos fallidos apareció un bug elegante:

**El problema:** `AuthService.login` es `@Transactional` . Cuando la contraseña era incorrecta, el código hacía `usuario.setIntentosFallidos(n + 1)` y después lanzaba `CredencialesInvalidasException` . Pero Spring, al ver una excepción unchecked, hacía **rollback de toda la transacción...** y con eso deshacía el incremento del contador.

Resultado: **el contador nunca subía y la cuenta nunca se bloqueaba**. Podías probar contraseñas infinitas.

La solución:

```
@Transactional(noRollbackFor = CredencialesInvalidasException.class)
```

Le dice a Spring: "*con esta excepción específica, no hagas rollback*". Así el incremento se guarda y la excepción igual llega al handler.

**El detalle de criterio que vale la pena mencionar:** se acotó a **esa única clase**, no a `RuntimeException` en general. Si se hubiera puesto genérico, **ninguna** excepción haría rollback y cualquier error dejaría datos a medio guardar. Se repasaron los 6 puntos de `AuthService` donde se lanza esa excepción para confirmar que ninguno deja una mutación parcial que debería revertirse.

## EL GlobalExceptionHandler — el traductor central

### Qué es

Una clase con `@RestControllerAdvice` . Esa anotación significa: "**esta clase escucha las excepciones de TODOS los controllers**". No hay que registrarla en ningún lado ni llamarla — Spring la conecta sola.

Adentro tiene métodos con `@ExceptionHandler(XxxException.class)` : cada uno atrapa un tipo y devuelve la respuesta correspondiente.

## La tabla completa de traducción

| Excepción atrapada                      | Status    | Mensaje devuelto                                                |
|-----------------------------------------|-----------|-----------------------------------------------------------------|
| RecursoNoEncontradoException            | 404       | El de la excepción                                              |
| ConflictoDeNegocioException             | 409       | El de la excepción                                              |
| ValidacionException                     | 400       | El de la excepción                                              |
| CredencialesInvalidasException          | 401       | El de la excepción + data                                       |
| MethodArgumentNotValidException         | 400       | Los errores de campo (ver abajo)                                |
| DataIntegrityViolationException         | 409 o 500 | Según el caso (ver abajo)                                       |
| HttpMessageNotReadableException         | 400       | "El cuerpo de la solicitud contiene datos con formato inválido" |
| MissingServletRequestParameterException | 400       | " <parámetro> : parámetro requerido ausente"                    |

Los 4 de arriba son las excepciones propias; los 4 de abajo son de Spring, y cada uno está ahí por un motivo concreto que vale la pena conocer.

## Handler 1: MethodArgumentNotValidException (400) — las validaciones de DTO

### Cuándo se dispara

Cuando `@Valid` sobre un `@RequestBody` encuentra que algún campo no cumple sus anotaciones.

### Qué devuelve

Dos cosas a la vez:

```
{
  "mensaje": "email: Ingresá un email con formato válido; password: La contraseña es obligatoria",
  "data": {
    "email": "Ingresá un email con formato válido",
    "password": "La contraseña es obligatoria"
  }
}
```

- El `mensaje` es todo junto en un string, como fallback.
- El `data` es un mapa `{campo: mensaje}`, para que el frontend pueda pintar cada error **debajo de su input** sin tener que parsear el texto.

### El bug de no determinismo — excelente material de mesa

Este handler tiene un bloque que a primera vista parece raro:

```
.sorted(Comparator.comparingInt(GlobalExceptionHandler::prioridadDeCampoObligatorio))
```

**El problema que resuelve:** un campo puede violar **dos anotaciones a la vez**. Por ejemplo, `@NotBlank` + `@Pattern`: si mandás un string vacío, falla el `@NotBlank` (está vacío) y el `@Pattern` (el vacío no matchea el regex).

Y acá está el detalle: **Hibernate Validator no garantiza en qué orden te devuelve las violaciones**. Sin ordenar, la misma petición podía devolver:

- una vez: *"El nombre es obligatorio"*  (el mensaje correcto)
- otra vez: *"El nombre solo puede contener letras"*  (confuso, porque está vacío, no mal escrito)

**No determinístico:** el mismo input daba mensajes distintos entre corridas. Un dolor de cabeza para depurar.

**La solución:** ordenar las violaciones poniendo primero las de tipo "obligatorio" ( `@NotBlank` , `@NotNull` , `@NotEmpty` ) y después usar `putIfAbsent`, que se queda con la primera de cada campo.

```
private static final List<String> CODIGOS_CAMPO_OBLIGATORIO =
    List.of("NotBlank", "NotNull", "NotEmpty");
```

Así, si el campo está vacío, siempre gana el mensaje "es obligatorio", sin importar el orden interno de Hibernate. Es la respuesta correcta desde el punto de vista del usuario: primero decile que falta, no que está mal escrito.

## Handler 2: DataIntegrityViolationException (409 o 500) — la red de seguridad de la base

### Cuándo se dispara

Cuando MySQL rechaza una operación porque viola una restricción: un `UNIQUE`, un `NOT NULL` o una `FOREIGN KEY`.

## Por qué existe: la condición de carrera

Este handler es la última línea de defensa contra un problema clásico llamado **check-then-act**:

```
Usuario A: "¿existe el email juan@mail.com?" → No
Usuario B: "¿existe el email juan@mail.com?" → No      (al mismo tiempo)
Usuario A: INSERT ...juan@mail.com                → OK
Usuario B: INSERT ...juan@mail.com                → ERROR: Duplicate entry
```

El service **chequea antes de insertar**, pero entre el chequeo y el insert puede meterse otro request. La única garantía real es el **unique** de la base de datos. Este handler traduce ese rechazo a un 409 legible, en vez de dejar pasar un 500 crudo.

## Lo inteligente: distingue tres tipos de violación

```
private static final int MYSQL_ERROR_DUPLICATE_ENTRY = 1062;

Throwable causaRaiz = ex.getMostSpecificCause();
boolean esViolacionUnique = causaRaiz instanceof SQLException sqlEx
    && sqlEx.getErrorCode() == MYSQL_ERROR_DUPLICATE_ENTRY;
```

`DataIntegrityViolationException` es un paraguas: envuelve violaciones de **unique**, de **not null** y de **fk**. Pero solo la primera es culpa del usuario:

| Violación                   | De quién es la culpa                                                 | Respuesta                                                             |
|-----------------------------|----------------------------------------------------------------------|-----------------------------------------------------------------------|
| <b>unique</b> (código 1062) | <b>Del usuario</b> — mandó un email/DNI/CUIT ya registrado           | <b>409</b> "Ya existe un registro con alguno de los datos ingresados" |
| <b>not null</b>             | <b>Del programador</b> — el service no completó un campo obligatorio | <b>500</b> + se loguea el error completo                              |
| <b>foreign key</b>          | <b>Del programador</b> — se apuntó a un id inexistente               | <b>500</b> + se loguea                                                |

**Por qué esta distinción importa:** si todo devolviera 409, un bug de programación quedaría disfrazado de "conflicto de datos" y **nadie se enteraría de que hay un error real**. Al mandar 500 y loguearlo ( `log.error(...)` ), el bug queda visible.

**Y de hecho esto atrapó un bug real:** en `DetallePedido.estado` faltaba asignar un valor obligatorio. Sin esta distinción, se habría visto como un 409 raro; con ella, saltó como 500 con el stack trace completo en el log.

**Detalle sobre el mensaje del 409:** dice "*alguno de los datos ingresados*", sin especificar cuál. Es a propósito — decir "ese email ya está registrado" le confirmaría a un atacante que esa cuenta existe. Es la misma idea que los mensajes ambiguos de recuperación de contraseña.

## Handler 3: `HttpMessageNotReadableException` (400) — el JSON malformado

### Cuándo se dispara

Cuando **Jackson no puede convertir el JSON** en el objeto Java. Ejemplos:

- Una fecha imposible: `"fechaNacimiento": "2005-02-30"` (30 de febrero no existe).
- Un texto donde se espera un número.
- Un valor de enum que no existe.
- JSON sintácticamente roto.

### Por qué hace falta manejarlo

Porque **Jackson corre ANTES que Bean Validation**. Si el JSON ni siquiera se puede convertir a objeto, nunca se llega a validar los campos — y por lo tanto `MethodArgumentNotValidException` nunca se dispara.

Sin este handler, ese error caía en la ruta genérica de Spring ( `/error` ) y podía terminar enmascarado como un 401 (ver el handler siguiente).

**Nota sobre el mensaje:** devuelve un texto genérico, sin el detalle técnico de Jackson. Eso es correcto: exponer el stack trace o los nombres internos de clases le da información de más a un atacante.

## Handler 4: `MissingServletRequestParameterException` (400) — el bug del 401 fantasma

### Cuándo se dispara

Cuando falta un `@RequestParam` obligatorio. El caso real: `GET /api/v1/geografia/localidades` sin el `?provinciaId=`.

### La historia del bug, que es la mejor de esta carpeta

Sin este handler pasaba lo siguiente:

- Spring detecta que falta el parámetro.
- Resuelve la excepción con `response.sendError(...)`.
- `sendError` hace que el contenedor haga un **forward interno a `/error`**.
- Ese **forward** vuelve a pasar por la cadena de filtros de Spring Security.

- Como `/error` no estaba en las rutas públicas, `anyRequest().authenticated()` lo interceptaba.
- Respuesta final: 401 Unauthorized**, en un endpoint público, por un parámetro faltante que debía dar 400.

Un error de "te falta un parámetro" apareciendo como "no estás autenticado". Imposible de diagnosticar sin entender la cadena completa.

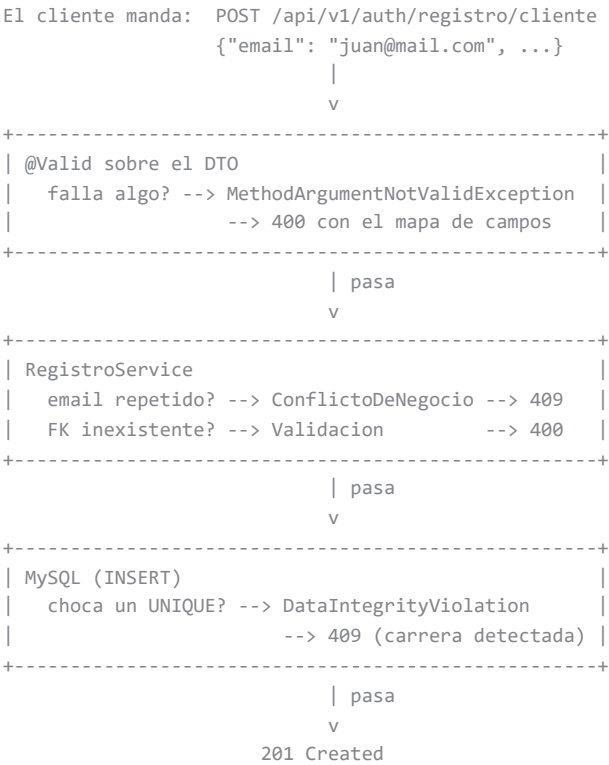
### La solución, en dos partes

- `/error` se agregó a `RUTAS_PUBLICAS` en `SecurityConfig` (ver `ESTUDIO-CONFIG.md`).
- Este handler atrapa la excepción y escribe la respuesta directo, evitando el forward por completo.

Son dos arreglos que se refuerzan: el handler evita el problema en este caso puntual, y `/error` público lo cubre para cualquier otro caso no manejado.

Este bug es una excelente respuesta si te preguntan "¿qué problema difícil resolviste?" — muestra que entendés cómo se ordena la cadena de filtros de Spring, que no es obvio.

## El flujo completo de un error



Fijate que hay tres capas de defensa para la misma regla (el email único): la anotación del DTO valida el formato, el service consulta si ya existe, y el `UNIQUE` de la base es la garantía final ante una condición de carrera. Eso se llama **defensa en profundidad**, y es una buena respuesta si te preguntan por robustez.

## Preguntas típicas de mesa

"¿Cómo manejas los errores?" Los services lanzan excepciones propias (4 tipos) y una clase con `@RestControllerAdvice` las traduce al código HTTP correspondiente. Los services no saben nada de HTTP.

"¿Por qué esas 4 excepciones y no más?" Porque cubren las 4 categorías reales de error de negocio: no existe (404), no se puede por el estado (409), el dato no sirve (400) y las credenciales fallan (401). Más granularidad no aportaría nada.

"¿Qué diferencia hay entre 400 y 409?" 400 es "lo que me mandaste está mal formado"; 409 es "está bien formado pero choca con el estado actual". Un 400 nunca va a funcionar; un 409 podría funcionar en otro momento.

"¿Qué pasa si dos personas se registran con el mismo email al mismo tiempo?" El service chequea antes, pero entre el chequeo y el insert hay una ventana. La garantía real es el `UNIQUE` de la base: el segundo insert falla, y el handler de `DataIntegrityViolationException` lo traduce a un 409 legible.

"¿Por qué todas extienden RuntimeException?" Para no ensuciar las firmas con `throws`, y porque Spring solo hace rollback automático de la transacción con excepciones unchecked.

"Contame un bug difícil que hayas resuelto." Un parámetro faltante en un endpoint público devolvía 401 en vez de 400, porque Spring hacía un forward interno a `/error` y ese forward volvía a pasar por el filtro de seguridad. Se arregló agregando `/error` a las rutas públicas y escribiendo un handler que evita el forward.

## Índice de archivos cubiertos en este documento

| Archivo                             | Status HTTP | Cuándo se dispara                                                                |
|-------------------------------------|-------------|----------------------------------------------------------------------------------|
| RecursoNoEncontradoException.java   | 404         | El recurso pedido no existe.                                                     |
| ConflictoDeNegocioException.java    | 409         | Choca contra una regla de negocio o el estado actual.                            |
| ValidacionException.java            | 400         | Validación de negocio que no cabe en una anotación de DTO.                       |
| CredencialesInvalidasException.java | 401         | Credenciales o token de un solo uso inválidos. Lleva <code>data</code> opcional. |
| GlobalExceptionHandler.java         | (varios)    | <code>@RestControllerAdvice</code> que traduce las 4 propias + 4 de Spring.      |